KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Course Recommendation and Skill Tracking System for Students

Submitted by

SANTHANAPRIYA K

25BIT052

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026 Semester: **I**

DECLARATION

I hereby declare that the project entitled " **Course Recommendation and Skill Tracking System for Students"** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of , Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

SANTHANAPRIYA K

(Your Name)

CERTIFICATE

This is to certify that the project entitled **Course Recommendation and Skill Tracking System for Students"** is a **bona fide work** carried out by **[SANTHANAPRIYA K] [I-BSc IT-.'A']**, [], **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date: 09.04.2026

Submitted for the Viva-Voce Examination held on **KG COLLAGE OF ARTS & SCIENCE**

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	

7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Course Recommendation and Skill Tracking System for Students** is a Java-based application designed to automate the process of managing inventory in warehouses. In many small businesses and storage facilities, inventory operations are carried out manually, which often leads to errors in stock calculation, delays in tracking products, and inefficient record management. This project aims to simplify inventory handling by providing an automated system that monitors stock levels and manages product details effectively.

The system allows users to add products, view available inventory, update stock quantities, and automatically identify items that fall below the reorder level. It ensures that low-stock products are detected in time, helping in proper stock replenishment and avoiding shortages. The application is developed using Core Java programming concepts such as classes, objects, methods, loops, conditional statements, and basic data structures like ArrayList.

The main objective of the system is to improve inventory management efficiency, reduce human errors, and provide accurate and real-time stock information. The system generates a clear view of inventory details including product ID, name, quantity, and reorder level, enabling better decision-making.

This project demonstrates the practical implementation of object-oriented programming concepts in Java and provides a strong foundation for developing more advanced inventory and warehouse management systems in the future.

1. INTRODUCTION

The Course Recommendation and Skill Tracking System is a console-based Java application designed to help students manage their skills and receive suitable course suggestions. In today's competitive world, students often struggle to identify which courses to pursue based on their current knowledge and interests. This system bridges that gap by providing a simple platform where students can input their skills and get relevant course recommendations.

The system also allows students to track their completed courses, helping them monitor their learning progress. By automating this process, the system reduces confusion and saves time. It is developed using Core Java concepts, making it simple, efficient, and easy to use.

This project is mainly focused on improving decision-making in education and encouraging continuous learning among students.

1.1 Objective

The main objectives of this system are:

- To help students identify suitable courses based on their skills
- To maintain a record of student skills and completed courses
- To provide a simple and user-friendly console interface
- To demonstrate the use of Java programming concepts in real-world applications
- To reduce manual effort in choosing learning paths

1.2 Overview

This system allows a student to:

- Create a profile
- Add skills (like Java, Python, Web Development)
- View their skill set
- Get recommended courses based on skills
- Track completed courses

The application uses Java classes, methods, and ArrayLists to store and manage data efficiently.

1.3 Features

- Easy student profile creation
- Add and view skills
- Automatic course recommendations
- Track completed courses
- Menu-driven console interface
- Simple and efficient design

2. PROGRAM ANALYSIS

Program analysis involves understanding the problem and defining how the system will solve it effectively.

2.1 Problem Definition

Students often face difficulty in selecting the right courses due to lack of guidance. Manual tracking of skills and courses can be confusing and inefficient. There is a need for a system that:

- Identifies student skills
- Suggests relevant courses
- Tracks progress

This project solves these problems using a simple automated system.

2.2 Project Overview

The system is designed as a console-based Java application. It consists of:

- A Student class to store data
- Methods to manage skills and courses
- A main menu for user interaction

The system processes user input and provides output based on logic and conditions.

2.3 Scopes and Limitations

Scope:

- Can be used by students for learning guidance
- Can be extended into web or mobile applications
- Useful for academic mini-projects

Limitations:

- Supports only one student at a time
- No database storage (data is temporary)

- Limited course recommendations

3. SYSTEM DESIGN

System design defines how the system components are organized and interact.

3.1 Modules

The system is divided into the following modules:

1. **Student Module**
    - **Stores student name, skills, and completed courses**
2. **Skill Management Module**
    - **Adds and displays skills**
3. **Recommendation Module**
    - **Suggests courses based on skills**
4. **Course Tracking Module**
    - **Tracks completed courses**
5. **User Interface Module**
    - **Menu-driven console interaction**

Each module works together to provide a complete system.

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the **Course Recommendation and Skill Tracking System for Students** , a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:

• Problem Analysis
• System Design
• Program Development
• Testing and Validation
• Result and Documentation

This section explains the steps followed to develop the system.

4.1 Problem Analysis

The problem was analyzed by identifying:

- **Student needs**
- **Required features**
- **Input and output requirements**

4.2 System Design

The system was designed using:

- **Class diagrams (conceptually)**
- **Modular approach**
- **Structured menu system**

4.3 Program Development

The program was developed using:

- **Java programming language**
- **Object-Oriented Programming concepts**
- **ArrayList for dynamic data storage**

4.4 Testing and Validation

The system was tested by:

- **Entering different inputs**
- **Checking output accuracy**
- **Verifying menu functionality**

All modules were tested individually and collectively.

4.5 Result and Documentation

The system successfully:

- **Stores student data**
- **Provides course recommendations**
- **Tracks completed courses**

Documentation was prepared to explain each part of the system clearly.

4.6 Algorithm

Step-by-step algorithm:

1. **Start the program**
2. **Display menu options**
3. **Accept user choice**
4. **If choice = create student → store name**
5. **If choice = add skill → store skill**
6. **If choice = view skills → display skills**
7. **If choice = recommend courses → show courses**
8. **If choice = complete course → store course**
9. **If choice = view completed → display courses**
10. **Repeat until exit**
11. **Stop**

4.7 Flowchart (Text Representation)

START

↓

Display Menu

↓

Enter Choice

↓

[Choice == 1] → Create Student

[Choice == 2] → Add Skill

[Choice == 3] → View Skills

[Choice == 4] → Recommend Courses

[Choice == 5] → Complete Course

[Choice == 6] → View Completed

[Choice == 7] → EXIT

↓

Repeat

↓

END

5. SOURCE CODE

```
import java.util.ArrayList;
import java.util.Scanner;


// Student class
class Student {
    String name;
    ArrayList<String> skills = new ArrayList<>();
    ArrayList<String> completedCourses = new ArrayList<>();


    Student(String name) {
```

```
        this.name = name;
    }

    void addSkill(String skill) {
        skills.add(skill);
        System.out.println("Skill added successfully!");
    }

    void viewSkills() {
        if (skills.isEmpty()) {
            System.out.println("No skills added yet.");
        } else {
            System.out.println("Your Skills:");
            for (String skill : skills) {
                System.out.println("- " + skill);
            }
        }
    }

    void recommendCourses() {
        System.out.println("\nRecommended Courses:");

        for (String skill : skills) {
            switch (skill.toLowerCase()) {
                case "java":
                    System.out.println("- Advanced Java Programming");
                    System.out.println("- Spring Boot Basics");
                    break;
                case "python":
```

```
                System.out.println("- Data Science with Python");
                System.out.println("- Machine Learning Basics");
                break;
            case "web development":
                System.out.println("- HTML, CSS, JavaScript");
                System.out.println("- React JS");
                break;
            case "c++":
                System.out.println("- Data Structures in C++");
                break;
            default:
                System.out.println("- General Programming Course");
        }
    }
}

void completeCourse(String course) {
    completedCourses.add(course);
    System.out.println("Course marked as completed!");
}

void viewCompletedCourses() {
    if (completedCourses.isEmpty()) {
        System.out.println("No completed courses.");
    } else {
        System.out.println("Completed Courses:");
        for (String course : completedCourses) {
            System.out.println("- " + course);
        }
```

```
        }
    }
}

// Main class
public class CourseRecommendationSystem {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Student student = null;

        while (true) {
            System.out.println("\n===== COURSE RECOMMENDATION SYSTEM
=====");
            System.out.println("1. Create Student");
            System.out.println("2. Add Skill");
            System.out.println("3. View Skills");
            System.out.println("4. Get Course Recommendations");
            System.out.println("5. Complete a Course");
            System.out.println("6. View Completed Courses");
            System.out.println("7. Exit");
            System.out.print("Enter choice: ");

            int choice = sc.nextInt();
            sc.nextLine(); // clear buffer

            switch (choice) {
                case 1:
                    System.out.print("Enter student name: ");
```

```
                    String name = sc.nextLine();
                    student = new Student(name);
                    System.out.println("Student created
successfully!");
                    break;

                case 2:
                    if (student == null) {
                        System.out.println("Create a student first!");
                        break;
                    }
                    System.out.print("Enter skill: ");
                    String skill = sc.nextLine();
                    student.addSkill(skill);
                    break;

                case 3:
                    if (student != null)
                        student.viewSkills();
                    else
                        System.out.println("Create a student first!");
                    break;

                case 4:
                    if (student != null)
                        student.recommendCourses();
                    else
                        System.out.println("Create a student first!");
                    break;
```

```
                case 5:
                    if (student == null) {
                        System.out.println("Create a student first!");
                        break;
                    }
                    System.out.print("Enter course name: ");
                    String course = sc.nextLine();
                    student.completeCourse(course);
                    break;

                case 6:
                    if (student != null)
                        student.viewCompletedCourses();
                    else
                        System.out.println("Create a student first!");
                    break;

                case 7:
                    System.out.println("Thank you!");
                    return;

                default:
                    System.out.println("Invalid choice!");
            }
        }
    }
}
```

6. INPUT AND OUTPUT SCREENS

```
===== COURSE RECOMMENDATION SYSTEM =====
1. Create Student
2. Add Skill
3. View Skills
4. Get Course Recommendations
5. Complete a Course
6. View Completed Courses
7. Exit
Enter choice: 1
Enter student name: santhanapriya
Student created successfully!
```

```
===== COURSE RECOMMENDATION SYSTEM =====
1. Create Student
2. Add Skill
3. View Skills
4. Get Course Recommendations
5. Complete a Course
6. View Completed Courses
7. Exit
Enter choice: 2
Enter skill: drawing
Skill added successfully!
```

```
===== COURSE RECOMMENDATION SYSTEM =====
1. Create Student
2. Add Skill
3. View Skills
4. Get Course Recommendations
5. Complete a Course
6. View Completed Courses
7. Exit
Enter choice: 3
Your Skills:
- drawing
```

```
===== COURSE RECOMMENDATION SYSTEM =====
1. Create Student
2. Add Skill
3. View Skills
4. Get Course Recommendations
5. Complete a Course
6. View Completed Courses
7. Exit
Enter choice: 4

Recommended Courses:
- General Programming Course
```

```
===== COURSE RECOMMENDATION SYSTEM =====
1. Create Student
2. Add Skill
3. View Skills
4. Get Course Recommendations
5. Complete a Course
6. View Completed Courses
7. Exit
Enter choice: 5
Enter course name: general programming course
Course marked as completed!
```

```
===== COURSE RECOMMENDATION SYSTEM =====
1. Create Student
2. Add Skill
3. View Skills
4. Get Course Recommendations
5. Complete a Course
6. View Completed Courses
7. Exit
Enter choice: 6
Completed Courses:
- general programming course
```

```
===== COURSE RECOMMENDATION SYSTEM =====
1. Create Student
2. Add Skill
3. View Skills
4. Get Course Recommendations
5. Complete a Course
6. View Completed Courses
7. Exit
Enter choice: 7
Thank you!
```

7. DISCUSSION & CONCLUSION

7.1 Discussion

The system was developed successfully using Java programming concepts. It simplifies the process of course selection and skill tracking. During development, various challenges such as input handling and data management were solved effectively.

The system demonstrates how programming can be applied to real-life problems in education.

7.2 Conclusion

The Course Recommendation and Skill Tracking System is a useful tool for students to manage their learning journey. It helps in identifying suitable courses and tracking progress efficiently. The project highlights the importance of automation and showcases the practical implementation of Java concepts. Future improvements can make it more powerful and user-friendly.

8. REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com